

Programmazione Android

Fondamenti

Prof. Marco Camurri

Riferimenti

Il contenuto di questa presentazione è frutto della traduzione e "semplificazione" del materiale presente nella seguente guida ufficiale:

<https://developer.android.com/guide/components/fundamentals>

Il sistema operativo

- Android è un **sistema operativo per dispositivi mobili** basato su una versione modificata del kernel di Linux
- Il sistema operativo "vero e proprio" è **open source** e **gratuito**. Il codice sorgente del progetto, noto come **AOSP** (Android Open Source Project) , è liberamente scaricabile dal sito <https://source.android.com/>
- La maggior parte dei dispositivi contiene anche **software proprietario** realizzato da Google, che NON rientra nel codice open source citato in precedenza
- Questo software proprietario include applicazioni (come Chrome o Google Play) e servizi*(come Google Mobile Services, Google Play Services)

*Possiamo pensare ai servizi come a librerie di "funzionalità aggiuntive" utilizzabili dalle applicazioni.

Il modello di sicurezza

- Android è ritenuto un sistema operativo sicuro (su cui è difficile installare *malware*)
- il **modello di sicurezza** di Android sfrutta i concetti di *utente* e di *permesso* già presenti nel kernel di Linux
- Ricorda infatti che:
 - Linux è un sistema multi-utente
 - Ogni file appartiene ad un particolare utente (il *proprietario*)
 - il proprietario o l'amministratore del sistema (*root*) possono, tramite il comando **chmod**, assegnare i permessi di lettura/scrittura ad un file in modo che esso sia accessibile solo dal proprietario e non dagli altri utenti

Il comando *chmod* di Linux

- Nell'esempio vengono assegnati i permessi di "lettura/scrittura/esecuzione" ("rwx") all'utente proprietario ("u") e il solo permesso di lettura ("r") agli altri ("o").

```
chmod u=rwx,g=rx,o=r myfile.txt
```

Il modello di sicurezza di Android

- Android sfrutta il meccanismo precedente generando un **nuovo utente Linux per ogni nuova app installata**
- Quindi ad ogni app corrisponde un utente diverso
- Quando un'app viene aperta, essa viene eseguita come se fosse stata lanciata dall'utente Linux ad essa corrispondente.
- In altre parole, l'app può compiere sole le azioni consentite a quell'utente (in base ai permessi che il sistema ha assegnato)

Il modello di sicurezza di Android

- Al momento dell'installazione di un'app il sistema crea una **cartella privata**, il cui nome corrisponde al "package name" dell'app, e assegna all'app solo i permessi necessari per leggere/scrivere in quella cartella (e in nessun'altra).
- Ogni app quindi è **isolata** rispetto alle altre: non può vedere o modificare il contenuto delle cartelle private delle altre app o del sistema operativo.
- Normalmente, sui dispositivi Android, l'utente root non è reso disponibile all'utilizzatore del dispositivo.
- Un dispositivo su cui ciò avviene è detto *rooted device* e il procedimento per ottenere l'accesso all'utente root è detto **rooting**.

Principio del minimo privilegio

- Ogni app, di default, riceve privilegi minimi, cioè può accedere solo alla porzione di sistema strettamente necessaria al proprio funzionamento; in pratica l'app può accedere solo ai file della propria cartella privata.
- L'app deve richiedere un **permesso** per accedere a dispositivi (come fotocamera, microfono, sensore GPS, schede di rete Bluetooth) o dati "comuni" (informazioni sui contatti, accesso ai file presenti nelle cartelle "pubbliche" del dispositivo, ecc..)
- I permessi possono essere concessi (**to grant**) automaticamente dal sistema operativo o dall'utilizzatore del dispositivo, a seconda del tipo di permesso richiesto (*normal* o *dangerous*).

App components

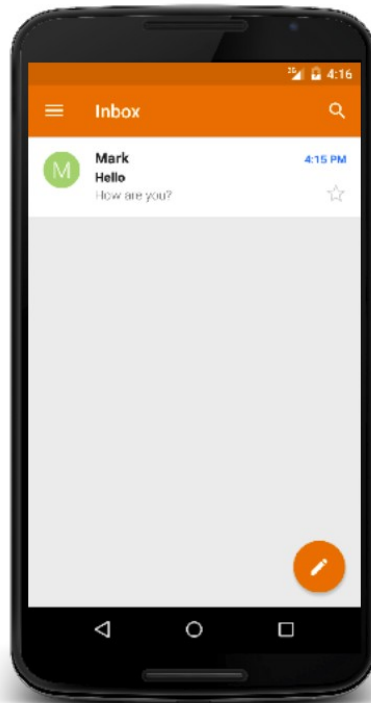
- Ogni app Android è composta da "blocchi di base" detti *app components* (sono i "mattoni di base" con cui costruire le app):
 - ***activity***
 - ***service***
 - ***broadcast receiver***
 - ***content provider***
- Un'app non deve per forza usare tutti e quattro i tipi di componente (le app più semplici di solito sono composte solo Activity)
- A ciascun tipo corrisponde una classe Java (Activity, Service, BroadcastReceiver, ContentProvider) che viene *estesa* dal programmatore

Activity

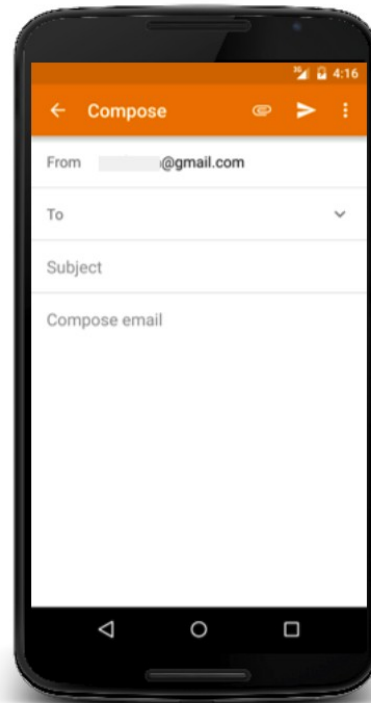
- Il componente **activity** rappresenta una **singola schermata** dell'applicazione
- Più formalmente, l'activity è definita dal framework come una *singola attività svolta dall'utente per mezzo di un'interfaccia grafica*.
- Questa definizione sottolinea il fatto che ad ogni activity dovrebbe corrispondere una ben precisa "finalità" dal punto di vista dell'utente
- Esempi di activity:
 - Activity per comporre una nuova e-mail
 - Activity per visualizzare l'elenco delle email ricevute
 - Activity per scattare una foto

Esempi di activity

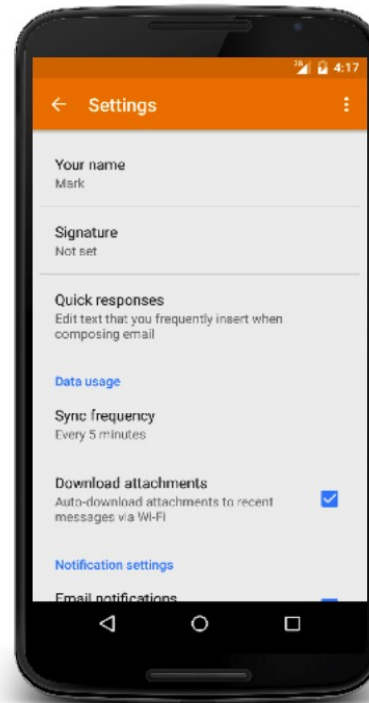
- Un'app di messaggistica potrebbe essere composta da diverse activity: 1) Lista delle conversazioni; 2) composizione di un nuovo messaggio; 3) Impostazioni;



Messages Activity



Compose Activity



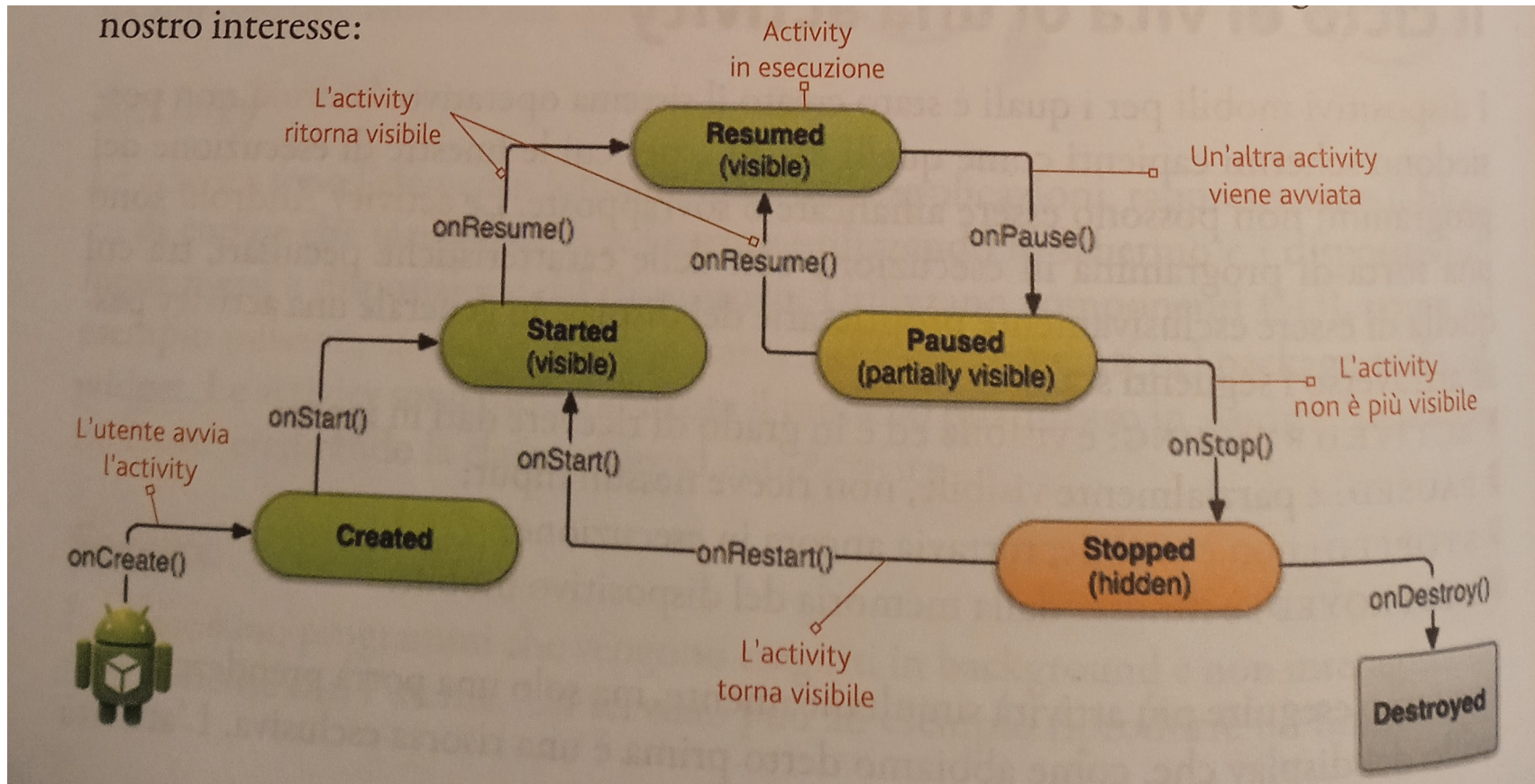
Settings Activity

Ciclo di vita di un'activity

- I dispositivi mobili non hanno schermi grandi per cui le finestre non possono essere affiancate o sovrapposte. Le Activity sono una specie di programma in esecuzione con delle caratteristiche particolari tra cui essere esclusivamente proprietarie del display.
- Stati di un'Activity:
 - **ACTIVE O RUNNING**: è visibile e in grado di ricevere dati in input
 - **PAUSED**: è parzialmente visibile, non riceve input
 - **STOPPED**: non è visibile ma è comunque in esecuzione
 - **DESTROYED**: è rimossa dalla memoria del dispositivo.

Si possono eseguire più attività simultaneamente ma solo una ha il controllo del display.

Ciclo di vita di un'Activity



Metodi Activity

protected void onCreate(android.os.Bundle savedInstanceState)

Viene richiamato alla creazione dell'attività, l'argomento `savedInstanceState` restituisce al metodo un eventuale stato dell'attività passato da un'altra istanza che è stata terminata, l'argomento vale `null` qualora non vi sia alcuno stato precedentemente salvato.

protected void onRestart()

Viene richiamato per segnalare che l'attività è stata riavviata dopo essere stata precedentemente arrestata.

protected void onStart()

Viene richiamato per segnalare che l'attività viene resa visibile sullo schermo.

protected void onResume()

Viene richiamato per segnalare che l'attività inizia l'interazione con l'utente.

protected void onPause()

Viene richiamato per segnalare che l'attività termina l'interazione con l'utente.

protected void onStop()

Viene richiamato per segnalare che l'attività non è più visibile sullo schermo.

protected void onDestroy()

Viene richiamato per segnalare che l'applicazione è stata terminata.

Metodi Activity

Per poter modificare il codice di questi metodi bisogna eseguire un **override** del metodo della classe padre, facendo attenzione ad inserire nella prima riga di codice di ciascuno di questi metodi il costruttore della classe base che si sta ridefinendo attraverso l'operatore **super**.

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        EdgeToEdge.enable(this);  
        setContentView(R.layout.activity_main);  
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {  
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());  
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);  
            return insets;  
        });  
    }  
}
```

Activity - Osservazioni importanti

Aspetti distintivi delle app Android (rispetto ai tradizionali programmi Desktop):

- Qualsiasi app installata sul sistema può richiedere l'attivazione (esecuzione) di un'activity di un'altra app del sistema. Questo meccanismo consente la **collaborazione** tra app dello stesso dispositivo.
- Esempio: un'app che necessita di scattare una foto può richiedere al S.O di avviare un'activity in grado di svolgere tale compito, anzichè creare da zero un'Activity che gestisce fotocamera.
- Mentre i programmi Desktop hanno un solo punto di ingresso ("main"), **in Android ogni app può avere più punti di ingresso** (multiple entry points). Ogni Activity è infatti un potenziale punto di avvio dell'app (vedi esempio slide successiva)

Punti d'ingresso multipli - Esempio

- Esempio: supponiamo che sul telefono sia installata un'app di posta elettronica (come Gmail).
- Quest'app sarà composta da un'activity principale che mostra l'elenco delle mail ricevute, e una seconda activity che consente di comporre una nuova mail.
- Cliccando sull'icona dell'app di posta si aprirà l'activity principale (punto di ingresso nr 1), ma se un'altra app avesse la necessità di inviare una mail, potrebbe decidere di avviare direttamente l'activity di composizione (punto di ingresso nr 2).

Il file *Manifest.xml*

Il ***manifest*** è un file xml in cui l'app dichiara tutte le informazioni necessarie al sistema operativo per installare l'app e per *integrarla* nel sistema. In questo file si trovano indicati:

- il nome dell'app
- il tema usato dall'app (stile grafico)
- l'icona dell'app
- l'elenco dei componenti dell'app (activity, service, broadcast receiver,..)
- i permessi richiesti dall'app
- azioni che l'app è in grado di svolgere (intent filters)